PROJE RAPORU: MODÜLER DOSYA İNDEKSLEME SİSTEMİ

Dersin Kodu ve Adı : BİL110 – PROGRAMLAMA VE HESAPLAMA II

Proje Adı : Dosya İndeksleme Sistemi

Ders Sorumlusu: Prof. Dr. Serdar Müldür

Hazırlayanlar : Hafsa Bera Pekcan – 250205001

Elif Casun – 250205014

Gülbade Yıldız - 250206003

Nuray Aliyeva – 250206901

1. Projenin Amacı ve Kapsamı

Bu projede, bilgisayarımızdaki temel dosya yönetim mantığını simüle eden, modüler yapıda bir sistem geliştirmeyi hedefledik. Temel amacımız, sadece dosyaları listelemek değil; aynı zamanda veriyi güvenli bir şekilde saklamak (kapsülleme), kullanıcı hatalarını minimize etmek (doğrulama) ve verilerin program kapandığında kaybolmamasını sağlamaktı.

Sistem, konsol tabanlı modüler bir yazılım olarak tasarlanmıştır. İçsel veri temsilleri kullanıcıdan gizlenmiştir (kapsülleme). İki temel Soyut Veri Tipi (ADT) kullanılmıştır: FileRecord ADT (tek bir dosyanın verileri) ve Dosya Yöneticisi ADT (dosya koleksiyonunun yönetimi).

2. Tasarım Yaklaşımımız: Neden ADT ve Kapsülleme?

Projeyi yazarken en çok dikkat ettiğimiz nokta Soyut Veri Tipleri (ADT) kullanımı oldu.

2.1 FileRecord ADT (Verinin Kimliği)

Bunu bir dosyanın "kimlik kartı" gibi düşündük. İçinde isim, uzantı, boyut ve tarih gibi bilgiler yer alıyor. Ancak bu bilgilere dışarıdan doğrudan müdahale edilmesini engelledik. Struct FileRecord tanımını .h dosyası yerine .c dosyasında gizleyerek, projenin başka yerlerinden bu verilere doğrudan ulaşılmasını engelledik.

```
// file_record.c içinde (dışarıdan gizli)
struct FileRecord {
    char name[101];
    char ext[11];
    double sizeMB;
    char createDate[20];
    char modDate[20];
};
```

Eğer bir dosyanın tarihi değişecekse, bunu ancak bizim yazdığımız setModDate fonksiyonu üzerinden yapılabilecek şekilde tasarladık. Bu da bizi beklenmedik bellek hatalarından koruyor.

2.2 Dosya Yöneticisi ADT (Sistemin Beyni)

Bu ise sistemin "beyni" konumunda. Listeyi yöneten, dosyaları ekleyip silen tüm mantığı burada gizledik. Kullanıcı sadece "dosya ekle" komutunu verir; arka planda listenin nasıl güncellendiğini bilmek zorunda değildir.

```
// file_manager.c içinde (dışarıdan gizli)
typedef struct Node {
    FileRecord* record;
    struct Node* next;
} Node;

struct FileManager {
    Node* head;
};
```

3. Veri Yapısı Seçimi: Neden "Bağlı Liste" (Linked List)?

Hocamızın sunduğu seçenekler (Liste, Yığın, Sıra) arasından Tek Yönlü Bağlı Liste yapısını seçmeye karar verdik.

Gerekçemiz: Bir dosya yönetim sisteminde kaç tane dosya olacağını önceden kestiremiyoruz. Eğer dizi (array) kullansaydık, başta sabit bir kapasite belirlememiz gerekecekti ve bu da belleği verimsiz kullanmak anlamına gelecekti. Bağlı liste sayesinde, her yeni dosyada bellekte sadece o dosyalık yer açıyoruz. Ayrıca dosya silerken dizilerdeki gibi tüm elemanları

kaydırmak zorunda kalmıyoruz; sadece işaretçilerin (pointer) yönünü değiştirerek işlemi tamamlıyoruz.

Kriter	Bağlı Liste	Dizi	Yığın	Sıra
Dinamik Boyut	✅ Çalışma anında büyür	❌ Sabit kapasite	✅	✅
Ekleme Maliyeti	✅ O(1) –Başa ekleme	❌ O(n) – Kaydırma gerekir	✅ O(1)	✅ O(1)
Silme Maliyeti	✅ O(1) –Pointer yönlendirme	❌ O(n) – Kaydırma gerekir	✅ O(1)	❌ O(1) sadece baş
Rastgele Erişim	⚠️ O(n) –Lineer arama	✅ O(1)	❌ LIFO uygun değil	❌ FIFO uygun değil
Bellek Verimliliği	✅ Sadece ihtiyaç kadar	❌ Önceden ayrılır	✅	✅

Yığın ve Sıra neden uygun değil? Yığın (Stack) LIFO prensibiyle çalışır; sadece son eklenen elemana erişilebilir. Sıra (Queue) FIFO prensibiyle çalışır; sadece ilk eklenen elemana erişilebilir. Dosya yöneti minde kullanıcı istediği dosyaya isme göre rastgele erişmek ister. Bu nedenle bağlı liste, hem esneklik hem de kapsülleme açısından en uygun seçimdir.

4. Algoritmik Analiz (Big-O)

Sistemin performansı üzerine yaptığımız analizler şu şekildedir:

4.1 Dosya Ekleme– O(n)

Aslında bir dosyayı listenin başına eklemek çok hızlıdır (O(1)). Ancak biz sistemin güvenliğini ön planda tuttuğumuz için, her ekleme öncesi “Acaba bu isimde başka bir dosya var mı?” diye tüm listeyi kontrol ediyoruz. Ayrıca saveToFile() fonksiyonu tüm listeyi dosyaya yazdığı için ek yük getiriyor.
Toplam: O(n) [isim kontrolü] + O(1) [boyut kontrolü] + O(1) [düğüm oluşturma] + O(n) [kaydetme] = O(n)

4.2 Dosya Silme– O(n)

Aradığımız dosya listenin en sonunda olabilir. Bu durumda listenin başından sonuna kadar tek tek bakmamız gerektiği için doğrusal bir zaman harcıyoruz. En iyi durum: O(1) (ilk düğümde bulunursa) En kötü durum: O(n) (son düğümde veya hiç yoksa) Genel olarak: O(n)

4.3 Dosya Arama– O(n)

Silme ile aynı mantıkta çalışır. strcmp() ile isim karşılaştırması yapıyoruz; bu işlem kısa isimler için O(1) gibi davransa da, en kötü durumda tüm liste dolaşılır.

4.4 Sıralama– O(n²)

Sistemde dosyaları boyutlarına göre küçükten büyüğe sıralamak amacıyla Selection Sort (Seçmeli Sıralama) algoritması tercih edilmiştir. Algoritma, bağlı listenin başından başlayarak her bir düğümü (i), kendisinden sonra gelen tüm düğümlerle (j) tek tek kıyaslar ve en küçük boyutlu dosyayı seçerek listenin başına doğru taşır.

4.5 Sistemdeki Tüm Dosyaları Listeleme - O(n²)

Bu fonksiyon çağrıldığında, bağlı listenin başından (mgr->head) başlanarak son düğüme kadar (curr = curr->next) doğrusal bir tarama gerçekleştirilir. Her bir düğümün üzerinden tam olarak bir kez geçilerek içindeki FileRecord verileri printRecord fonksiyonu vasıtasıyla ekrana tablo formatında yazdırılır. İşlem hacmi ve çalışma süresi sistemdeki toplam dosya sayısı (n) ile doğru orantılı olarak arttığı için zaman karmaşıklığı O(n) düzeyindedir.

4.6 Dosya Değiştirme Tarihini Güncelleme - O(n)

Tarihi güncellenecek olan dosyanın tespiti için bağlı liste üzerinde en baştan itibaren doğrusal bir arama (Linear Search) gerçekleştirilir. En kötü senaryoda (aranan dosyanın listenin en sonunda olması veya sistemde hiç bulunmaması durumunda) tüm liste (n eleman) taranacağı için arama maliyeti **O(n)** olur. Dosya bulunduğunda, string kopyalama (strcpy) ile gerçekleştirilen setModDate operasyonu ise sabit zamanlı (O(1)) bir işlemdir. Bu iki durum birleştiğinde fonksiyonun toplam zaman karmaşıklığı **O(n)** olmaktadır.

4.7 Sistem Toplam Kapasitesini Hesaplama - O(n)

Sistemde statik bir boyut sayacı tutulmadığı için, toplam doluluk oranının hesaplanması amacıyla listenin başından sonuna kadar bir for döngüsü işletilir. Her bir düğüme uğranarak dosya boyutları (getFileSizeMB) tek tek çekilir ve kümülatif bir toplam değişkenine eklenir. Döngünün adım sayısı doğrudan bağlı listedeki düğüm sayısına (n) bağlı olduğundan, operasyon doğrusal zamanda, yani **O(n)** karmaşıklığında tamamlanır.

4.8 Tüm Listeyi Hafızadan Kalıcı Olarak Temizleme - O(n)

Bu operasyon, dinamik olarak ayrılan belleğin sisteme güvenli bir şekilde iade edilmesini (Memory Leak/Bellek sızıntısının önlenmesini) sağlar. Bir while döngüsü yardımıyla bağlı listedeki her bir vagon (düğüm) tek tek ziyaret edilir; bir sonraki düğümün adresi kaybolmasın diye geçici bir işaretçide saklanarak mevcut düğüm ve içindeki FileRecord yapısı free() komutuyla bellekten tamamen silinir. Silinecek düğüm sayısı toplam dosya sayısı (n) kadar olduğundan, bu temizleme işleminin zaman karmaşıklığı **O(n)** olarak hesaplanmıştır.

5. Karşılaştığımız Zorluklar ve Çözümler (Test Süreci)

Test aşamasında özellikle “uç durumlar” (edge cases) üzerinde çok durduk ve sistemin her koşulda kararlı çalışmasını sağladık.

5.1 Normal Vakalar Testi

Sistemin kararlılığını ölçmek için uç durumlardan önce, tüm girdilerin kurallara tam uyduğu ve sistemin kesintisiz çalıştığı "Normal Vakalar Testi" simüle edilmiştir. Bu test kapsamında, kullanıcının her adımı hatasız girdiğinde sistemin döngülere takılmadan, doğrudan veri tabanına yazma ve dinamik listeye ekleme adımlarını başarıyla tamamladığı doğrulanmıştır.

Örnek: Menüden 1 seçin. İsim olarak rapor, uzantı olarak pdf, boyut olarak 15.5 ve tarih olarak 15.05.2026 girin. Dosyanın başarıyla eklendiğini göreceksiniz.

5.2 Hatalı Tarihler– isValidDate Kontrolü

Kullanıcının 31 Şubat veya 13. ay gibi imkansız tarihler girmesini engellemek için tarih kontrol fonksiyonuna artık yıl (leap year) kontrolü ekledik. daysInMonth dizisi ile her ayın gün sınırını tuttuk ve artık yıl kurallarını (4'e bölünebilirlik, 100'e bölünemezlik, 400'e bölünebilirlik) uyguladık.

Test: 29.02.2024 (artık yıl) → Kabul edildi

Test: 29.02.2023 (artık yıl değil) → Reddedildi

Test: 31.02.2026 (geçersiz gün) → Reddedildi

5.3 Yasaklı Karakterler ve İsim Güvenliği– isValidName Kontrolü

Modern işletim sistemlerindeki standartlara uyum sağlayarak; /, \, :, *, ?, ", <, >, | gibi karakterlerin dosya isminde kullanılmasını engelledik. Ayrıca dosya isimlerinin 100 karakteri aşması durumunda sistemin hata vermesini sağlayarak bellek taşmalarının önüne geçtik.

5.4 Yinelenen Kayıt Senaryoları

addFile fonksiyonu içerisinde her yeni kayıttan önce tüm listeyi tarayan bir kontrol mekanizması kurduk. Bu sayede sistemde aynı isimle ikinci bir dosya oluşturulmasını engelleyerek veri bütünlüğünü koruduk.

5.5 Kapasite Yönetimi ve Boyut Sınırları

Sisteme toplam 8GB (MAX_TOTAL_SYSTEM_LIMIT) sınırı koyduk. Ayrıca tek bir dosyanın 1GB sınırını aşmasını engelleyerek hem bireysel dosya boyutunu hem de toplam disk doluluğunu kontrol altına aldık.

Test: 8*(1GB) + 200MB ekleme → İlk aşamada sırayla 8 tane 1gb dosya eklendiği zaman sorun olmaz ancak sonrasında herhangi bir boyutlu dosya eklemesinde "Kapasite aşımı" hatası verir.

5.6 Uç Durum ve Boş Yapı Testleri

Sistemde hiç dosya yokken listeleme, arama veya silme komutları verildiğinde programın çökmediğini; bunun yerine kullanıcıya "Liste boş" veya "Bulunamadı" gibi anlamlı uyarılar verdiğini doğruladık.

İşlem	Boş Liste Durumu	Beklenen Çıktı
Listeleme	`mgr->head == NULL`	"Liste bos."
Arama	Dosya bulunamadı	"[HATA] Bulunamadi."
Silme	Dosya bulunamadı	"[HATA] Bulunamadi."
Sıralama	`mgr->head == NULL`	Hızlıca döner (return)
Kapasite	`calculateTotalSize() = 0`	%0.00 doluluk

5.7 Dosya Kalıcılığı ve Veri Kurtarma

Verilerin program kapandığında kaybolmaması için veriler.txt dosyası üzerinden bir sistem kurduk. saveToFile ve loadFromFile fonksiyonları sayesinde, her açılışta eski kayıtların otomatik olarak yüklen mesini ve her işlemden sonra verilerin güncel halinin kaydedilmesini sağladık. Format: isim|uzanti|boyut|olusturma_tarih|degisiklik_tarihi

Örnek: belge|pdf|1024mb|12.12.2007|12.12.2007

6. Sonuç

Bu proje, modüler programlamanın ve veriyi gizlemenin (kapsülleme) büyük projelerde karmaşıklığı nasıl azalttığını ekip olarak görmemizi sağladı. Bağlı liste yapısını kullanarak esnek bir sistem kurduk ve gerçek hayat senaryolarına (dosya silme, güncelleme, sıralama) uygun fonksiyonlar geliştirdik.

Kapsülleme sayesinde FileRecord ve FileManager yapılarının iç detaylarını değiştirsek bile main.c dosyasını yeniden derlemek zorunda kalmayız. Big-O analizi bize sistemin sınırlarını gösterdi; özellikle Bubble Sort'un $O(n^2)$ karmaşıklığı ile büyük ölçekli sistemlerde daha verimli algoritmalara ihtiyaç duyulacağını öğrendik.

Ortaya çıkan ürün; hatalara karşı dirençli, veriyi koruyan ve gerçek hayat senaryolarına cevap veren bir dosya yönetim sistemidir.